



LeetCode 1365 — How Many Numbers Are Smaller Than the Current Number

1. Problem Title & Link

Title: LeetCode 1365: How Many Numbers Are Smaller Than the Current Number

Link: <https://leetcode.com/problems/how-many-numbers-are-smaller-than-the-current-number/>

2. Problem Statement (Short Summary)

Given an integer array `nums`, for every element find **how many numbers in the array are smaller than it**. Return the count for every position.

3. Examples (Input → Output)

Example 1

Input:

`nums = [8,1,2,2,3]`

Output:

`[4,0,1,1,3]`

Explanation:

8 → 4 numbers smaller (1,2,2,3)

1 → 0 numbers smaller

2 → 1 number smaller (1)

2 → 1 number smaller (1)

3 → 3 numbers smaller (1,2,2)

Example 2

Input:

`nums = [6,5,4,8]`

Output:

`[2,1,0,3]`

Example 3

Input:

`nums = [7,7,7,7]`

Output:



[0,0,0,0]

4. Constraints

$2 \leq \text{nums.length} \leq 500$

$0 \leq \text{nums}[i] \leq 100$

Important Observation:

Values are very small (0 to 100)

This enables counting-based solutions.

5. Core Concept (Pattern / Topic)

Sorting

HashMap

Counting / Frequency Array

6. Thought Process (Step-by-Step Explanation)

Brute Force

For each number:

Count how many numbers are smaller than it.

Example:

8 → compare with every element

1 → compare with every element

2 → compare with every element

This requires:

$n \times n$ comparisons

Time Complexity:

$O(n^2)$

Optimized Thinking

If array is sorted:

[1,2,2,3,8]

The first occurrence index tells how many numbers are smaller.

1 → index 0 → 0 smaller

2 → index 1 → 1 smaller



3 → index 3 → 3 smaller

8 → index 4 → 4 smaller

Store this mapping in a HashMap.

7. Visual / Intuition Diagram (ASCII)

Original

[8,1,2,2,3]

Sorted

[1,2,2,3,8]

Index

0 1 2 3 4

Map

1 → 0

2 → 1

3 → 3

8 → 4

Result:

8 → 4

1 → 0

2 → 1

2 → 1

3 → 3

Answer:

[4,0,1,1,3]

8. Pseudocode

copy nums into temp

sort temp

store first occurrence index in hashmap

for each num in nums



```
        answer[i] = hashmap[num]

    return answer
```

9. Code Implementation

Python

```
class Solution:
    def smallerNumbersThanCurrent(self, nums):
        sorted_nums = sorted(nums)

        rank = {}

        for i, num in enumerate(sorted_nums):
            if num not in rank:
                rank[num] = i

        return [rank[num] for num in nums]
```

Java

```
import java.util.*;

class Solution {

    public int[] smallerNumbersThanCurrent(int[] nums) {

        int[] sorted = nums.clone();

        Arrays.sort(sorted);

        HashMap<Integer,Integer> map = new HashMap<>();

        for(int i = 0; i < sorted.length; i++) {

            map.putIfAbsent(sorted[i], i);
        }

        int[] ans = new int[nums.length];

        for(int i = 0; i < nums.length; i++) {

            ans[i] = map.get(nums[i]);
        }

        return ans;
    }
}
```



10. Time & Space Complexity

Sorting Approach

Time:

$O(n \log n)$

Sorting dominates.

Space:

$O(n)$

HashMap + answer array.

11. Common Mistakes / Edge Cases

Mistake 1

Using the last occurrence index.

Wrong:

2 → index 2

Correct:

2 → first index 1

Mistake 2

Forgetting duplicates.

Mistake 3

Modifying original array accidentally.

12. Detailed Dry Run

Input:

nums = [8,1,2,2,3]

Sorted:

[1,2,2,3,8]

HashMap:

Number	Smaller Count
1	0
2	1
3	3
8	4

Build answer:



nums[i]	count
8	4
1	0
2	1
2	1
3	3

Result:

[4,0,1,1,3]

13. Common Use Cases (Real-Life / Interview)

Ranking Systems

Student rank calculation

Leaderboard systems

Competition ranking

Score comparison

14. Common Traps

Trap 1

Using last occurrence instead of first occurrence.

Trap 2

Not handling duplicates correctly.

Trap 3

Updating map every time.

Use:

map.putIfAbsent()

15. Builds To (Related LeetCode Problems)

LC 1365 Smaller Numbers Than Current

LC 217 Contains Duplicate



LC 242 Valid Anagram

LC 347 Top K Frequent Elements

LC 49 Group Anagrams

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	$O(n^2)$	$O(1)$	Easy
Sorting + Map	$O(n \log n)$	$O(n)$	Common
Counting Array	$O(n)$	$O(101)$	Best for this problem

Counting Array Optimization

Since:

$0 \leq \text{nums}[i] \leq 100$

Use frequency array of size 101.

Achieves:

Time : $O(n)$

Space : $O(101)$

17. Why This Solution Works

After sorting, the first occurrence position of a number equals the count of numbers smaller than it.

Using a HashMap allows retrieving this count in $O(1)$.

Therefore every answer is computed correctly.

18. Variations / Follow-Up Questions

1. Solve without sorting.
2. Solve in $O(n)$.
3. What if values are up to 10^9 ?
4. What if duplicates are not allowed?
5. Return ranks instead of counts.